R6.A.05 TP3

Étape 1 — Création des microservices

1 - Installations

On installe Node.js sur **https://nodejs.org** :

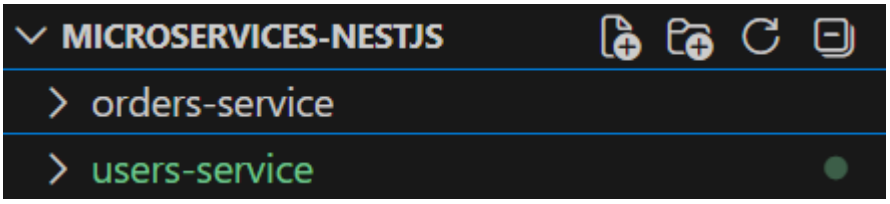
```
C:\Users\edoua>node -v
v24.14.0

C:\Users\edoua>npm -v
11.9.0
```

On installe NestJS CLI :

```
C:\Users\edoua>npm i -g @nestjs/cli

added 213 packages in 19s

38 packages are looking for funding
  run `npm fund` for details
npm notice
npm notice New minor version of npm available! 11.9.0 -> 11.11.0
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.11.0
npm notice To update run: npm install -g npm@11.11.0
npm notice

C:\Users\edoua>nest --version
11.0.16

C:\Users\edoua>
```

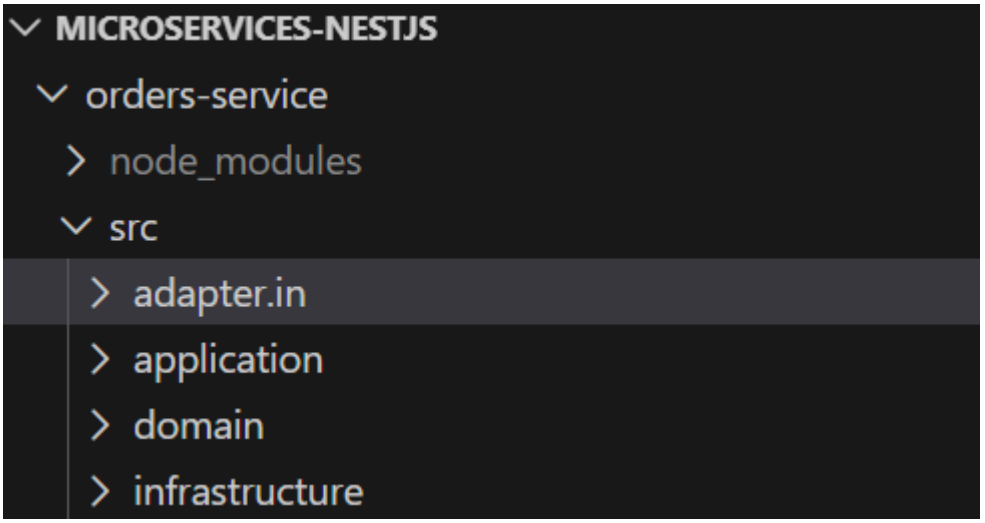
On crée nos deux projets :

```
>> nest new orders-service - UPEC\Documents\BUT3\S6\R6.A.05 Développement Avancé\TP3\microservices-NestJS>
CREATE orders-service/package.json (2056 bytes)
CREATE orders-service/README.md (5126 bytes)
CREATE orders-service/tsconfig.build.json (101 bytes)
CREATE orders-service/tsconfig.json (702 bytes)
CREATE orders-service/src/app.controller.ts (286 bytes)
CREATE orders-service/src/app.module.ts (259 bytes)
CREATE orders-service/src/app.service.ts (150 bytes)
CREATE orders-service/src/main.ts (236 bytes)
CREATE orders-service/src/app.controller.spec.ts (639 bytes)
CREATE orders-service/test/jest-e2e.json (192 bytes)
CREATE orders-service/test/app.e2e-spec.ts (694 bytes)

✓ Installation in progress... 🍵

🚀  Successfully created project orders-service
👉  Get started with the following commands:

$ cd orders-service
$ npm run start


                    Thanks for installing Nest 🙏
            Please consider donating to our open collective
                   to help us maintain this package.


               🍷  Donate: https://opencollective.com/nest

PS C:\Users\edoua\OneDrive - UPEC\Documents\BUT3\S6\R6.A.05 Développement Avancé\TP3\microservices-NestJS>
```



2 - Création de l’architecture Clean

On crée nos dossiers :



3 - On implémente les DTOs

On crée un fichier create-user.dto.ts :

```
TS create-user.dto.ts 1 ●
users-service > src > adapter.in > dto > TS create-user.dto.ts > CreateUserDto > role
1   import { IsString, IsEmail, IsNotEmpty } from 'class-validator';
2
3   export class CreateUserDto {
4     @IsString()
5     @IsNotEmpty()
6     nom: string;
7
8     @IsEmail()
9     email: string;
10
11    @IsString()
12    @IsNotEmpty()
13    role: string;
14  }
```

Un fichier user-response.dto :

```
TS user-response.dto.ts ×
users-service > src > adapter.in > dto > TS user-re
1   export class UserResponseDto {
2     id: string;
3     nom: string;
4     email: string;
5     role: string;
6   }
```

On fait de même pour la modification d'un user, la création et la modification d'un order et la réponse à une requête concernant les orders.

4 - Configurer l'injection de dépendances via les modules NestJS

On crée un fichier [users.modules.ts]() dans lequel on défini les chemins vers nos fichiers :

```
TS users.module.ts ×
users-service > src > users > TS users.module.ts > UsersModule
1   import { Module } from '@nestjs/common';
2   import { UsersController } from '../adapter.in/users.controller';
3   import { UsersService } from '../application/users.service';
4   import { UsersRepository } from '../infrastructure/users.repository';
5
6   @Module({
7     controllers: [UsersController],  // ← reçoit les requêtes HTTP
8     providers: [
9       UsersService,                  // ← logique métier
10      UsersRepository,               // ← accès aux données
11    ],
12  })
13  export class UsersModule {}
```

On modifie notre fichier [app.modules.ts]() pour lier nos modules :

```
TS app.module.ts ×
users-service > src > TS app.module.ts > ...
1   import { Module } from '@nestjs/common';
2   import { UsersModule } from './users/users.module';
3
4   @Module({
5     imports: [UsersModule],
6   })
7   export class AppModule {}
```

Étape 2 — Fonctionnalités à implémenter

users-service :

Dans un premier temps, nous implémentons une entité représentant l'utilisateur qui sera manipulée en interne par l'application. C'est par son biais que seront transmises les données entre le controller, le repository et le service. On crée donc un fichier [user.entity.ts]() dans domain :

```
TS app.module.ts    TS user.entity.ts ×
users-service > src > domain > TS user.entity.ts > User
1   export class User {
2     id: string;
3     nom: string;
4     email: string;
5     role: string;
6
7     constructor(id: string, nom: string, email: string, role: string) {
8       this.id = id;
9       this.nom = nom;
10      this.email = email;
11      this.role = role;
12    }
13  }
```

Ensuite nous créons un controller qui va récupérer les requêtes HTTP. On crée un fichier users.controller.ts dans adapter.in :

```
TS app.module.ts    TS users.module.ts    TS users.controller.ts ×    TS user.entity.ts
users-service > src > adapter.in > TS users.controller.ts > UsersController
17  @Controller('users')
18  export class UsersController {
19    constructor(private readonly usersService: UsersService) {}
20
21    @Get()
22    findAll(): UserResponseDto[] {
23      return this.usersService.findAll();
24    }
25
26    @Get(':id')
27    findById(@Param('id') id: string): UserResponseDto {
28      return this.usersService.findById(id);
29    }
30
31    @Post()
32    create(@Body() dto: CreateUserDto): UserResponseDto {
33      return this.usersService.create(dto);
34    }
```

On crée ensuite users.service.ts dans application qui s'occupera de la logique métier :

```
TS app.module.ts   TS users.module.ts   TS users.controller.ts   TS users.service.ts ×   TS user.entit
users-service > src > application > TS users.service.ts > UsersService
8   export class UsersService {
15    findById(id: string): User {
17      if (!user) throw new NotFoundException(`Utilisateur ${id} introuvable`);
18      return user;
19    }
20
21    create(dto: CreateUserDto): User {
22      return this.usersRepository.create(dto.nom, dto.email, dto.role);
23    }
24
25    update(id: string, dto: UpdateUserDto): User {
26      const user = this.usersRepository.update(id, dto);
27      if (!user) throw new NotFoundException(`Utilisateur ${id} introuvable`);
28      return user;
29    }
30
31    delete(id: string): void {
32      const deleted = this.usersRepository.delete(id);
33      if (!deleted) throw new NotFoundException(`Utilisateur ${id} introuvable`);
34    }
35  }
```

On crée enfin un fichier users.repository.ts dans infrastructure qui s'occupe de stocker les données :

```
TS users.repository.ts ×
users-service > src > infrastructure > TS users.repository.ts > UsersRepository > create
1   import { Injectable } from '@nestjs/common';
2   import { User } from '../domain/user.entity';
3   import { v4 as uuidv4 } from 'uuid';
4
5   @Injectable()
6   export class UsersRepository {
7     private users: User[] = []; // stockage en mémoire pour l'instant
8
9     findAll(): User[] {
10      return this.users;
11    }
12
13    findById(id: string): User | undefined {
14      return this.users.find(user => user.id === id);
15    }
16
17    create(nom: string, email: string, role: string): User {
18      const user = new User(uuidv4(), nom, email, role);
19      this.users.push(user);
20      return user;
21    }
```

Dans nos DTOs, nous avons des décorateurs pour chaque attributs qui définissent des conditions sur les données (nom doit être un string par exemple). Pour que class validator vérifie automatiquement que les données passées dans le dto via la requête HTTP correspondent bien il faut ajouter cette ligne dans main.ts :

```
TS users.repository.ts    TS main.ts ×
users-service > src > TS main.ts > bootstrap
1   import { NestFactory } from '@nestjs/core';
2   import { AppModule } from './app.module';
3   import { ValidationPipe } from '@nestjs/common';
4
5   async function bootstrap() {
6     const app = await NestFactory.create(AppModule);
7     app.useGlobalPipes(new ValidationPipe());
8     await app.listen(process.env.PORT ?? 3000);
9   }
10  bootstrap();
```

Aussi, il faut importer class validator pour que le code puisse compiler :
npm install class-validator class-transformer uuid

orders-service:

On applique la même logique pour orders-service. Nous créons un controller, un service, un repository, une entité utilisée en interne, on injecte les dépendances via les modules NestJS et on connecte class validator. Attardons nous cependant sur la spécificité des commandes en comparaison des utilisateurs, la règle métier contraignant le statut de la commande. En effet, cette dernière ne peut pas passer à SHIPPED si elle est toujours à CREATED et non pas à PAID. Cette règle est une règle métier et sera donc la responsabilité du service. Il faudra, dans la fonction de mise à jour de la commande, vérifier cette condition:

```
27  updateStatus(id: string, dto: UpdateOrderStatusDto): Order {
28    const order = this.findById(id);
29
30    // Règle métier : ne peut pas être SHIPPED si pas PAID
31    if (dto.status === OrderStatus.SHIPPED && order.status !== OrderStatus.PAID) {
32      throw new CannotShipUnpaidOrderException();
33    }
34
35    const updated = this.ordersRepository.updateStatus(id, dto.status);
36    if (!updated) throw new NotFoundException(`Commande ${id} introuvable`);
37    return updated;
38  }
```

On crée alors une exception personnalisée dans domain :

```
TS order.exceptions.ts ×
orders-service > src > domain > TS order.exceptions.ts > CannotShipUnpaidOrderException
1  import { BadRequestException } from '@nestjs/common';
2
3  export class CannotShipUnpaidOrderException extends BadRequestException {
4    constructor() {
5      super('Une commande ne peut pas être expédiée si elle n\'est pas payée');
6    }
7  }
```

Étape 3 — Communication entre services

Explication de l'API Gateway

Pour l'étape 3 on doit créer un troisième service NestJS qui va servir de gateway, c'est lui qui va recevoir toutes les requêtes et les rediriger vers le bon service. On installe axios pour pouvoir faire des requêtes HTTP, on crée le controller qui gère les redirections et on configure le main.ts pour qu'il puisse lire le body des requêtes. Les trois services doivent tourner en même temps sur des ports différents, la gateway sur le 3002, users sur le 3000 et orders sur le 3001.

Mise en place

Création du répertoire et du service api gateway

```
PS D:\CODE\TP3 R6.A.5 nust js> cd .\microservices-NestJS\
PS D:\CODE\TP3 R6.A.5 nust js\microservices-NestJS> nest new api-gateway
 ✨  We will scaffold your app in a few seconds..

 ✓ Which package manager would you ❤ to use? npm
CREATE api-gateway/.prettierrc (56 bytes)
CREATE api-gateway/eslint.config.mjs (934 bytes)
CREATE api-gateway/nest-cli.json (179 bytes)
CREATE api-gateway/package.json (2053 bytes)
CREATE api-gateway/README.md (5126 bytes)
CREATE api-gateway/tsconfig.build.json (101 bytes)
CREATE api-gateway/tsconfig.json (702 bytes)
CREATE api-gateway/src/app.controller.ts (286 bytes)
CREATE api-gateway/src/app.module.ts (259 bytes)
CREATE api-gateway/src/app.service.ts (150 bytes)
CREATE api-gateway/src/main.ts (236 bytes)
CREATE api-gateway/src/app.controller.spec.ts (639 bytes)
CREATE api-gateway/test/jest-e2e.json (192 bytes)
CREATE api-gateway/test/app.e2e-spec.ts (694 bytes)

 ✓ Installation in progress... 🍷

 🚀  Successfully created project api-gateway
 👉  Get started with the following commands:

 $ cd api-gateway
 $ npm run start


                    Thanks for installing Nest 🙏
            Please consider donating to our open collective
                   to help us maintain this package.


               🍷  Donate: https://opencollective.com/nest

PS D:\CODE\TP3 R6.A.5 nust js\microservices-NestJS>
```

On installe aussi axios pour gérer la communication REST entre les microservices. Axios est une librairie qui permet de faire des requêtes HTTP (GET, POST, etc.) depuis Node.js,

comme un fetch mais plus simple à utiliser. Déjà utilisé durant mon alternance, c'est plus simple et centralisé que les simples fetch (Matéo).

```
PS D:\CODE\TP3 R6.A.5 nust js\microservices-NestJS> cd .\api-gateway\
PS D:\CODE\TP3 R6.A.5 nust js\microservices-NestJS\api-gateway> npm install axios

  added 3 packages, and audited 703 packages in 2s

  147 packages are looking for funding
    run `npm fund` for details

  8 moderate severity vulnerabilities

  To address issues that do not require attention, run:
    npm audit fix

  To address all issues (including breaking changes), run:
    npm audit fix --force

  Run `npm audit` for details.
PS D:\CODE\TP3 R6.A.5 nust js\microservices-NestJS\api-gateway>
```

On peut maintenant commencer à mettre en place l'api gateway, pour cela on crée un controller : gateway.controller.ts. Il interceptera toutes les requêtes HTTP entrantes et les redirigera vers le bon microservice selon l'URL : les routes /users* seront transmises au users-service (port 3000) et les routes /orders* au orders-service (port 3001), via axios, en conservant la méthode, le body et les headers originaux.

Explication du controller :

1. mise en place

On déclare la classe comme controller NestJS sans préfixe d'URL pour intercepter toutes les routes + les imports (axios pour requêtage et les types Express pour les requêtes/réponses

```
import { All, Controller, Req, Res } from '@nestjs/common';
import type { Request, Response } from 'express';
import axios from 'axios';

@Controller()
export class GatewayController {
```

@All intercepte toutes les méthodes HTTP (GET, POST, PUT, DELETE...) pour toute URL commençant par /users et pareil pour /orders

On retransmet la requête telle quelle au users-service sur le port 3000 et on renvoie sa réponse au client. Pareil pour les orders sauf que c'est sur le port 3001

```
@All('users/*path')
async proxyUsers(@Req() req: Request, @Res() res: Response) {
  try {
    const response = await axios({
      method: req.method,
      url: `http://localhost:3000${req.url}`,
      data: req.body,
      headers: req.headers,
    });
    res.status(response.status).json(response.data);
  } catch (error) {
    res.status(502).json({ message: 'Service indisponible' });
  }
```

Et si le service est inaccessible, on renvoie une erreur 502 au lieu de laisser la gateway crasher.

2. correction du controller

Pour que la gateway puisse rediriger les requêtes, on crée gateway.controller.ts . On a du mettre 4 méthodes au lieu de 2 car au début on avait juste users/*path mais ça interceptait pas /users tout seul, donc on a rajouté une méthode pour la route de base. On a aussi eu un problème de timeout en passant tous les headers originaux, on a donc décidé de transmettre uniquement le content type en plus :

```
@All('users')
async proxyUsersRoot(@Req() req: Request, @Res() res: Response) {
  try {
    const response = await axios({
      method: req.method,
      url: `http://localhost:3000${req.url}`,
      data: req.body,
      headers: { 'Content-Type': 'application/json' },
    });
    res.status(response.status).json(response.data);
  } catch (error) {
    res.status(502).json({ message: 'Service indisponible' });
  }
}
```

Configuration du controller

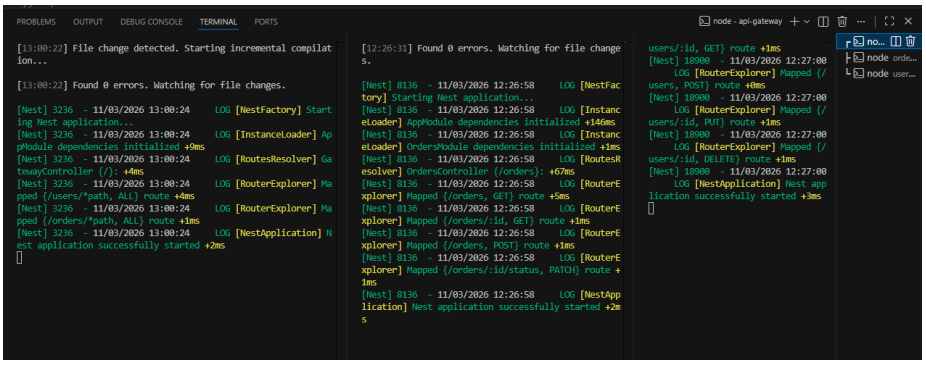
On déclare ensuite le controller dans app.module.ts pour que NestJS sache qu'il doit l'utiliser. Sans cette déclaration, le controller serait ignoré et aucune requête ne serait interceptée. On configure également main.ts pour activer la lecture du body des requêtes, car sans cela le body d'un POST serait vide et ne serait jamais transmis au microservice cible.

```
async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  app.use(json());
  app.use(urlencoded({ extended: true }));
  await app.listen(3002);
}
bootstrap();
```

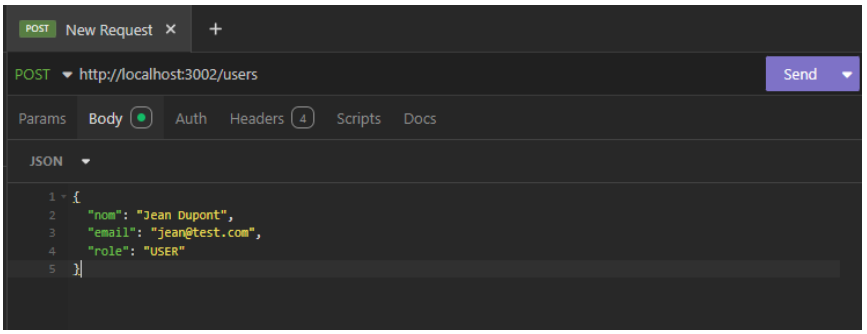
Tests

Voici un test rapide pour voir si l'api gateway fonctionne correctement :

1. on lance le service en parallèle des deux autres services avec npm run start:dev :



2. On peut tester via postman ou insomnia dans mon cas :



configuration de la requête sur users en passant par l'api gateway pour créer un utilisateur

3. La réponse de la requête :

l'utilisateur à bien été créé.

Étape 4 — Sécurité (JWT)

Explication de JWT

Pour mettre en place la sécurité JWT, on installe les dépendances nécessaires uniquement dans les deux services et pas dans la gateway, car c'est les services eux-mêmes qui vérifient les tokens, la gateway fait juste rediriger les requêtes. C'est le users-service qui sera responsable de générer les tokens JWT lors de la connexion, car c'est lui qui gère les utilisateurs et l'authentification. Les deux services devront ensuite vérifier que le token est valide avant d'exécuter quoi que ce soit.

Le flux est le suivant : le client envoie ses identifiants au users-service qui lui retourne un token, le client utilise ensuite ce token dans toutes ses requêtes vers les deux services, et chacun vérifie de son côté que le token est valide.

Pour ce TP, l'authentification est simulée avec des identifiants fixes (admin/password123). Dans un projet réel, on irait vérifier les identifiants en base de données avec un mot de passe hashé. L'important ici est de démontrer le fonctionnement du mécanisme JWT.

Installation, configuration et création de l'endpoint de récupération

Dans les deux services, installe les dépendances JWT :

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                                                    powersh

PS D:\CODE\TP3 R6.A.5 nust js> cd .\microservices-NestJS\users-service\
PS D:\CODE\TP3 R6.A.5 nust js\microservices-NestJS\users-service> npm install @nestjs/jwt @nestjs/passport passport passport-jwt

added 21 packages, and audited 728 packages in 3s

148 packages are looking for funding
  run `npm fund` for details

8 moderate severity vulnerabilities

To address issues that do not require attention, run:
  npm audit fix

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.
PS D:\CODE\TP3 R6.A.5 nust js\microservices-NestJS\users-service> npm install -D @types/passport-jwt

added 3 packages, and audited 731 packages in 2s

148 packages are looking for funding
  run `npm fund` for details

8 moderate severity vulnerabilities

To address issues that do not require attention, run:
  npm audit fix

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.
PS D:\CODE\TP3 R6.A.5 nust js\microservices-NestJS\users-service>
```

(même commande pour orders)

Création de authController avec un utilisateur test (admin) dans le répertoire du service users

```
croservices-NestJS > users-service > src > adapter.in > TS auth.controller.ts > AuthController > login
import { Controller, Post, Body, UnauthorizedException } from '@nestjs/common';
import { JwtService } from '@nestjs/jwt';

class LoginDto {
  username: string;
  password: string;
}

@Controller('auth')
export class AuthController {
  constructor(private readonly jwtService: JwtService) {}

  @Post('login')
  login(@Body() dto: LoginDto) {
    // Simulation d'authentification
    if (dto.username !== 'admin' || dto.password !== 'password123') {
      throw new UnauthorizedException('Identifiants invalides');
    }

    const payload = { username: dto.username, role: 'admin' };
    return {
      access_token: this.jwtService.sign(payload),
    };
  }
}
```

et configuration du controller dans users.module.ts :

```
@Module({
  imports: [
    JwtModule.register({
      secret: process.env.JWT_SECRET || 'secret',
      signOptions: { expiresIn: '24h' },
    }),
  ],
  controllers: [UsersController, AuthController],
  providers: [UsersService, UsersRepository],
  exports: [JwtModule],
})
export class UsersModule {}
```

Test dans insomnia de l'utilisateur test admin :

POST New Request ×

POST http://localhost:3000/auth/login Send

Params Body Auth Headers 4 Scripts Docs

JSON

```
{
  "username": "admin",
  "password": "password123"
}
```

201 Created 43 ms 192 B Just Now

Preview Headers 7 Cookies Tests 0 / 0 → Mock Console

Preview


```
{
  "access_token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImFkbWluIiwicm9sZSI6ImFkbWluIiwiaWF0IjoxNzczMjMyODEyLCJleHAiOjE3NzMzMTkyMTJ9.NlfWkIFbZbNAcypWVg2CiFuDnxsSWgfURN7moitESCU"
}
```


Ajout dans api gateway la récupération du token pour centraliser, on a ajouté la route de /auth dans le controller :

```
@All('auth/*path')
async proxyAuth(@Req() req: Request, @Res() res: Response) {
  try {
    const response = await axios({
      method: req.method,
      url: `http://localhost:3000${req.url}`,
      data: req.body,
      headers: {
        'Content-Type': 'application/json',
        'Authorization': req.headers.authorization,
      },
    });
    res.status(response.status).json(response.data);
  } catch (error) {
    res.status(502).json({ message: 'Service indisponible' });
  }
}
```

Création d'un guard qui sera appelé sur les routes pour vérifier le token

Pour protéger les routes, on crée un guard jwt.guard.ts dans chaque service. Son rôle est d'intercepter chaque requête avant qu'elle atteigne le controller et de vérifier que le token JWT est présent dans le header Authorization. Si le token est absent ou invalide, le guard renvoie une erreur 401 et la requête n'est jamais traitée. Si le token est valide, il laisse passer la requête et injecte les informations de l'utilisateur dedans.

```
@Injectable()
export class JwtAuthGuard implements CanActivate {
  constructor(private readonly jwtService: JwtService) {}

  canActivate(context: ExecutionContext): boolean {
    const request = context.switchToHttp().getRequest<Request>();
    const token = request.headers.authorization?.split(' ')[1];

    if (!token) {
      throw new UnauthorizedException('Token manquant');
    }

    try {
      const payload = this.jwtService.verify(token, {
        secret: process.env.JWT_SECRET || 'secret',
      });
      request['user'] = payload;
      return true;
    } catch {
      throw new UnauthorizedException('Token invalide');
    }
  }
}
```

Ajout sur toutes les routes

Dans le service users

Maintenant il faut appliquer le guard sur toutes les routes des deux services. Dans le users.controller.ts, on ajoute le guard sur le controller :

```
import { UseGuards } from '@nestjs/common';
import { JwtAuthGuard } from '../guards/jwt.guard';
```

et configurer le guard sur les routes, pour cela on le met juste sur le controller qui va automatiquement le mettre sur les routes :

```
@UseGuards(JwtAuthGuard)
@Controller('users')
export class UsersController {
  constructor(private readonly usersService: UsersService) {}
```

Dans le service orders (on répère les mêmes étapes)

On copie le fichier guard dans orders :

et on reconfigure aussi le fichier orders.module.ts :

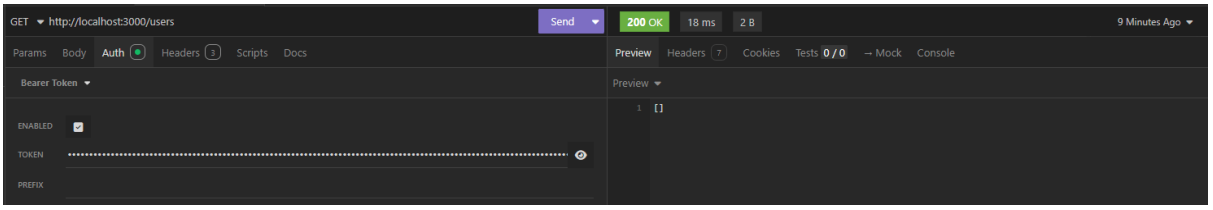
```
import { Module } from '@nestjs/common';
import { JwtModule } from '@nestjs/jwt';
import { OrdersController } from '../adapter.in/orders.control
import { OrdersService } from '../application/orders.service';
import { OrdersRepository } from '../infrastructure/orders.rep

@Module({
  imports: [
    JwtModule.register({
      secret: process.env.JWT_SECRET || 'secret',
      signOptions: { expiresIn: '24h' },
    }),
  ],
  controllers: [OrdersController],
  providers: [OrdersService, OrdersRepository],
})
export class OrdersModule {}
```

dans le controller des routes on peut ensuite comme pour users, ajouter le jwt comme pour users :

```
import { UseGuards } from '@nestjs/common';
import { JwtAuthGuard } from '../guards/jwt.guard';

@UseGuards(JwtAuthGuard)
@Controller('orders')
export class OrdersController {
  constructor(private readonly ordersService: OrdersService) {}
```

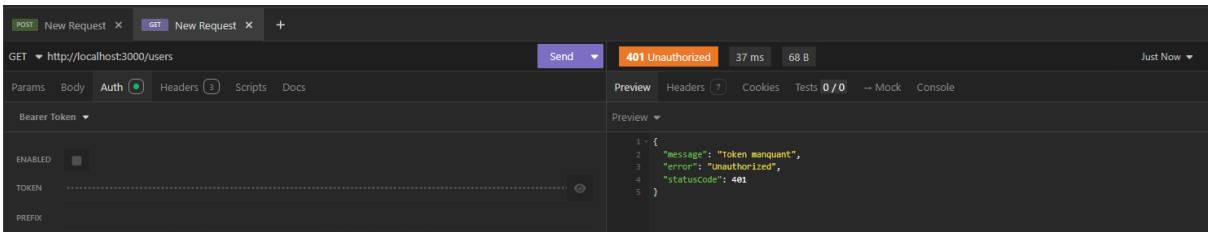
Test des routes avec le token

C’est bien valide



Test des routes sans le token

On est bien refusé :



Étape 5 — Documentation

On ajoute une documentation Swagger pour chaque service de nos services. Cette documentation sera visible à l'url /api-docs. Nous détaillerons les étapes pour users-service, les étapes sont identiques pour orders-service.

D'abord nous ajoutons les dépendances :
npm install @nestjs/swagger swagger-ui-express

Ensuite, nous configurons Swagger dans main.ts :

```
TS main.ts M ×   TS auth.controller.ts M   TS users.controller.ts M
users-service > src > TS main.ts > bootstrap > config
1  import { NestFactory } from '@nestjs/core';
2  import { AppModule } from './app.module';
3  import { ValidationPipe } from '@nestjs/common';
4  import { DocumentBuilder, SwaggerModule } from '@nestjs/swagger';
5
6  async function bootstrap() {
7    const app = await NestFactory.create(AppModule);
8    app.useGlobalPipes(new ValidationPipe());
9
10   // Configuration Swagger
11   const config = new DocumentBuilder()
12     .setTitle('Users Service')
13     .setDescription('API de gestion des utilisateurs')
14     .setVersion('1.0')
15     .addBearerAuth()
16     .build();
17
18   const document = SwaggerModule.createDocument(app, config);
19   SwaggerModule.setup('api-docs', app, document);
20
21   await app.listen(process.env.PORT ?? 3000);
22 }
23 bootstrap();
24
```

Pour chaque DTO, on ajoute pour chaque attribut le décorateur @ApiProperty avec une valeur d'exemple qui sera préremplie dans les tests de requêtes Swagger :

TS create-user.dto.ts M × | TS update-user.dto.ts M | TS users.controller.ts M

users-service > src > adapter.in > dto > TS create-user.dto.ts > ...

```typescript
import { IsString, IsEmail, IsNotEmpty } from 'class-validator';
import { ApiProperty } from '@nestjs/swagger';

export class CreateUserDto {
  @ApiProperty({ example: 'Edouard Peyrouty' })
  @IsString()
  @IsNotEmpty()
  nom: string;

  @ApiProperty({ example: 'edouard@gmail.com' })
  @IsEmail()
  email: string;

  @ApiProperty({ example: 'admin' })
  @IsString()
  @IsNotEmpty()
  role: string;
}
```

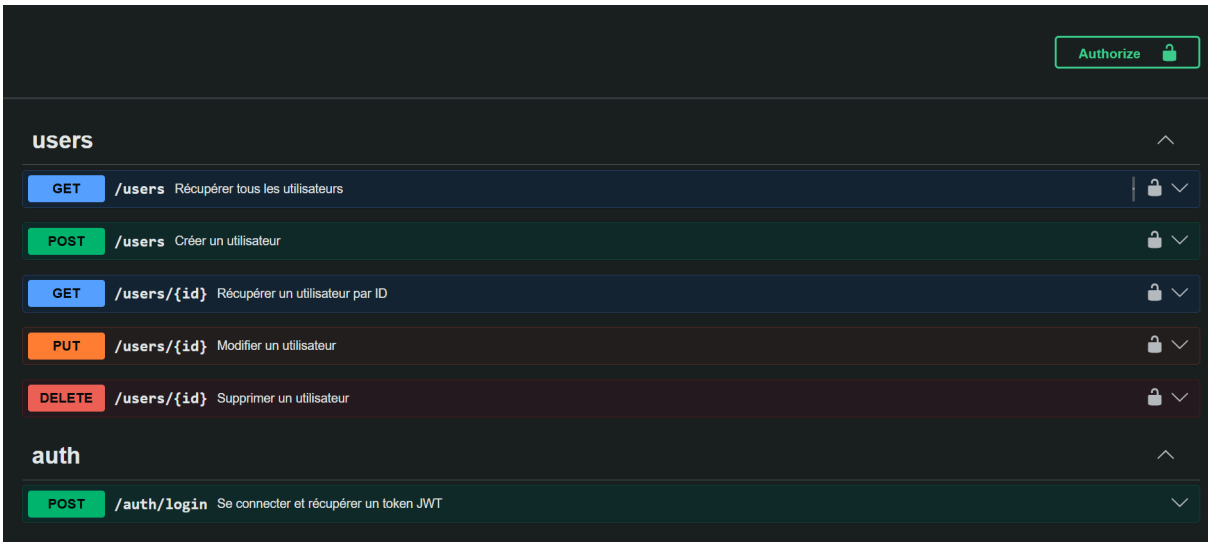
On ajoute enfin de la documentation dans le controler :

- On groupe les endpoints par nom de route
- On définit une description pour chaque endpoint
- On précise les sorties possibles

Pour users.controler.ts :

```
23  @ApiTags('users')        // ← groupe les routes sous "users" dans Swagger
24  @ApiBearerAuth()
25  @UseGuards(JwtAuthGuard)
26  @Controller('users')
27  export class UsersController {
28    constructor(private readonly usersService: UsersService) {}
29
30    @Get()
31    @ApiOperation({ summary: 'Récupérer tous les utilisateurs' })
32    @ApiResponse({ status: 200, type: [UserResponseDto] })
33    findAll(): UserResponseDto[] {
34      return this.usersService.findAll();
35    }
36
37    @Get(':id')
38    @ApiOperation({ summary: 'Récupérer un utilisateur par ID' })
39    @ApiResponse({ status: 200, type: UserResponseDto })
40    @ApiResponse({ status: 404, description: 'Utilisateur introuvable' })
41    findById(@Param('id') id: string): UserResponseDto {
42      return this.usersService.findById(id);
43    }
```

Résultat sur http://localhost:3000/api-docs :



Étape 6 — Conteneurisation

1 - Dockerfile

Pour chacun des trois services on crée un Dockerfile qui copie le code source et expose le port approprié. Exemple pour users-service :

```
Dockerfile users-service ×   docker-c
users-service > Dockerfile > ...
1  FROM node:24-alpine
2  WORKDIR /app
3  COPY package*.json ./
4  RUN npm install
5  COPY . .
6  RUN npm run build
7  EXPOSE 3000
8  CMD ["node", "dist/main"]
```

2 - Docker compose

On crée ensuite un fichier docker-compose.yml pour lancer nos deux service ainsi que notre gateway facilement :

```
docker-compose.yml ×   Dockerfile o
docker-compose.yml
1  version: '3.9'
2
   Run All Services
3  services:
     Run Service
4    users-service:
5      build: ./users-service
6      ports:
7        - "3000:3000"
8
     Run Service
9    orders-service:
10     build: ./orders-service
11     ports:
12       - "3001:3001"
13
     Run Service
14   api-gateway:
15     build: ./api-gateway
16     ports:
17       - "3002:3002"
18     depends_on:
19       - users-service
20       - orders-service
```

Étape 7 — Tests et vérification

1 - Tests unitaires Jest

Ici, l'objectif est de tester les cas d'usage métier et la bonne implémentation des règles métiers. A ce titre, nous allons avant tout tester les services. Attardons-nous sur l'exemple du service des utilisateurs, la même logique étant appliquée au service des commandes. On crée donc un fichier users.service.spec.ts qu'on exécutera pour faire nos tests :

- On commence par générer un faux repository dont on va forcer les valeurs de retour des fonctions (on appelle cela des mocks). L'objectif ici est de tester le comportement du service en fonction de ce que retourne le repository et non pas de tester le repository. Cela permet aussi de rendre le test indépendant de l'implémentation du repository et valable même si le repository change :

```
10  beforeEach(async () => {
11    // On crée un faux repository (mock) pour ne pas dépendre du vrai
12    const module: TestingModule = await Test.createTestingModule({
13      providers: [
14        UsersService,
15        {
16          provide: UsersRepository,
17          useValue: {
18            findAll: jest.fn(),
19            findById: jest.fn(),
20            create: jest.fn(),
21            update: jest.fn(),
22            delete: jest.fn(),
23          },
24        },
25      ],
26    }).compile();
27
28    service = module.get<UsersService>(UsersService);
29    repository = module.get<UsersRepository>(UsersRepository);
30  });
```

- Pour chaque cas d'usage, on vérifie le comportement du service pour chaque sortie possible du repository. Ici, pour la récupération d'un utilisateur en fonction de son id, le service doit renvoyé l'utilisateur renvoyé par le repository, ou, si le repository ne renvoie rien, levé une exception :

```
75  describe('findById', () => {
76    it('doit retourner un utilisateur existant', () => {
77      const fakeUser = { id: 'uuid-1', nom: 'Edouard', email: 'edouard@test.com', role: 'admin' };
78      jest.spyOn(repository, 'findById').mockReturnValue(fakeUser);
79
80      const result = service.findById('uuid-1');
81      expect(result).toEqual(fakeUser);
82    });
83
84    it('doit lancer une NotFoundException si l\'utilisateur n\'existe pas', () => {
85      jest.spyOn(repository, 'findById').mockReturnValue(undefined);
86
87      expect(() => service.findById('uuid-inexistant')).toThrow(NotFoundException);
88    });
89  });
```

- Pour orders-service on teste notre règle métier lorsque l'on modifie le statut d'une commande :

```
117    it('doit bloquer le passage CREATED → SHIPPED', () => {
118      const fakeOrder = { id: 'uuid-1', userId: 'u1', products: [], status: OrderStatus.CREATED };
119
120      jest.spyOn(repository, 'findById').mockReturnValue(fakeOrder);
121
122      expect(() =>
123        service.updateStatus('uuid-1', { status: OrderStatus.SHIPPED })
124      ).toThrow(CannotShipUnpaidOrderException);
125    });
```

- On ajoute les lignes suivantes dans packages.json pour des raisons de compatibilités avec uuid :

```
{} package.json M ×   TS orders.service.spec.ts U
users-service > {} package.json > {} jest
38    "devDependencies": {
63      "typescript-eslint": "^8.20.0"
64    },
65    "jest": {
66      "moduleFileExtensions": [
67        "js",
68        "json",
69        "ts"
70      ],
71      "rootDir": "src",
72      "testRegex": ".*\\.spec\\.ts$",
73      "transform": {
74        "^.+\\.(t|j)s$": "ts-jest"
75      },
76      "transformIgnorePatterns": [
77        "node_modules/(?!(uuid)/)"
78      ],
79      "collectCoverageFrom": [
80        "**/*.(t|j)s"
81      ],
82      "coverageDirectory": "../coverage",
83      "testEnvironment": "node"
84    }
85  }
```

- On peut enfin exécuter nos tests :

```
PS C:\Users\edoua\OneDrive - UPEC\Documents\BUT3\S6\R6.A.05 Développement Avancé\TP3\microservices-NestJS> cd orders-service
>> npm run test

> orders-service@0.0.1 test
> jest

 PASS  src/app.controller.spec.ts
 PASS  src/application/orders.service.spec.ts

Test Suites: 2 passed, 2 total
Tests:       11 passed, 11 total
Snapshots:   0 total
Time:        3.78 s
Ran all test suites.
PS C:\Users\edoua\OneDrive - UPEC\Documents\BUT3\S6\R6.A.05 Développement Avancé\TP3\microservices-NestJS\orders-service>
```

```
PS C:\Users\edoua\OneDrive - UPEC\Documents\BUT3\S6\R6.A.05 Développement Avancé\TP3\microservices-NestJS\users-service> npm run
 test

> users-service@0.0.1 test
> jest

 PASS  src/app.controller.spec.ts
 PASS  src/application/users.service.spec.ts

Test Suites: 2 passed, 2 total
Tests:       10 passed, 10 total
Snapshots:   0 total
Time:        3.14 s
Ran all test suites.
PS C:\Users\edoua\OneDrive - UPEC\Documents\BUT3\S6\R6.A.05 Développement Avancé\TP3\microservices-NestJS\users-service>
```

2 - Tests d'intégration

On va désormais tester les endpoint rest à l'aide de requêtes http qui seront simulées par supertest qui est intégré à jest. Ici, on va utiliser les vrais repositories des services et tester le code ainsi que le contenu des réponses de l'API. On crée un fichier app.e2e.spec.ts:

- On teste l'authentification :

```
// ── AUTH ──────────────────────────────────────────

describe('POST /auth/login', () => {
  it('doit retourner un token JWT avec les bons identifiants', async () => {
    const response = await request(app.getHttpServer())
      .post('/auth/login')
      .send({ username: 'admin', password: 'password123' })
      .expect(201);

    expect(response.body.access_token).toBeDefined();
    jwtToken = response.body.access_token; // on garde le token pour les tests suivants
  });

  it('doit retourner 401 avec de mauvais identifiants', async () => {
    await request(app.getHttpServer())
      .post('/auth/login')
      .send({ username: 'admin', password: 'mauvais' })
      .expect(401);
  });
});
```

- On teste si le service refuse l'accès lorsque l'on a pas le token :

```
// ── USERS SANS TOKEN ──────────────────────────────

describe('Authentification JWT', () => {
  it('doit refuser l\'accès sans token (401)', async () => {
    await request(app.getHttpServer())
      .get('/users')
      .expect(401);
  });

  it('doit refuser l\'accès avec un token invalide (401)', async () => {
    await request(app.getHttpServer())
      .get('/users')
      .set('Authorization', 'Bearer tokeninvalide')
      .expect(401);
  });
});
```

- On vérifie chaque endpoint :

```
describe('POST /users', () => {
  it('doit créer un utilisateur', async () => {
    const response = await request(app.getHttpServer())
      .post('/users')
      .set('Authorization', `Bearer ${jwtToken}`)
      .send({ nom: 'Edouard', email: 'edouard@test.com', role: 'admin' })
      .expect(201);

    expect(response.body.id).toBeDefined();
    expect(response.body.nom).toBe('Edouard');
    createdUserId = response.body.id; // on garde l'id pour les tests suivants
  });

  it('doit refuser une création avec un email invalide (400)', async () => {
    await request(app.getHttpServer())
      .post('/users')
      .set('Authorization', `Bearer ${jwtToken}`)
      .send({ nom: 'Edouard', email: 'pasunmail', role: 'admin' })
      .expect(400);
  });
});
```

- On peut exécuter nos tests :

```
PS C:\Users\edoua\OneDrive - UPEC\Documents\BUT3\S6\R6.A.05 Développement Avancé\TP3\microservices-NestJS\users-service> npm run
 test

> users-service@0.0.1 test
> jest

 PASS  src/app.controller.spec.ts
 PASS  src/application/users.service.spec.ts
 PASS  src/app.e2e.spec.ts

Test Suites: 3 passed, 3 total
Tests:       24 passed, 24 total
Snapshots:   0 total
Time:        4.789 s
Ran all test suites.
PS C:\Users\edoua\OneDrive - UPEC\Documents\BUT3\S6\R6.A.05 Développement Avancé\TP3\microservices-NestJS\users-service>
```

```
PS C:\Users\edoua\OneDrive - UPEC\Documents\BUT3\S6\R6.A.05 Développement Avancé\TP3\microservices-NestJS\orders-service> npm run test

> orders-service@0.0.1 test
> jest

 PASS  src/app.controller.spec.ts
 PASS  src/application/orders.service.spec.ts
 PASS  src/app.e2e.spec.ts

Test Suites: 3 passed, 3 total
Tests:       20 passed, 20 total
Snapshots:   0 total
Time:        4.446 s
Ran all test suites.
PS C:\Users\edoua\OneDrive - UPEC\Documents\BUT3\S6\R6.A.05 Développement Avancé\TP3\microservices-NestJS\orders-service>
```

Les tests précédents ne passent pas par la gateway mais par les services directement. Pour tester le passage par la gateway, supertest ne suffit plus. Il faudra faire des vraies requêtes http en utilisant axios et pour cela lancer les deux services ainsi que la gateway chacun dans son terminal puis le test dans un 4ème terminal. C'est donc plus lourd mais le test n'étant exécuté que ponctuellement, il peut être pertinent de laisser cette possibilité. On crée alors un fichier gateway.e2e.spec.ts dans lequel on regroupe tous les tests effectuer précédemment. La différence étant que l'on ne fait plus les requêtes avec la fonction **request** de supertest mais avec axios :

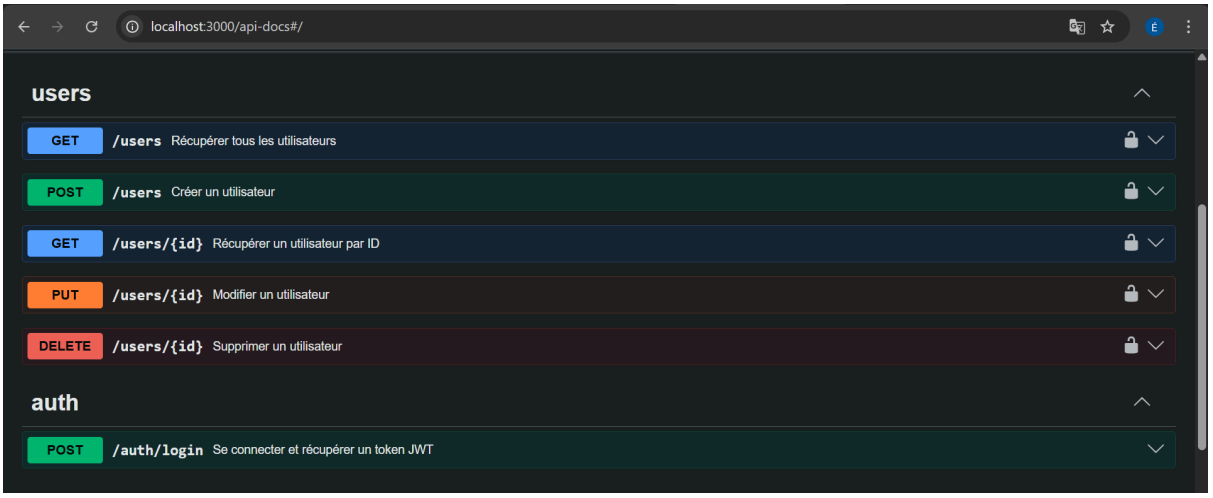
```
10  // ── AUTH ──────────────────────────────────────────
11
12  describe('POST /auth/login', () => {
13    it('doit retourner un token JWT avec les bons identifiants', async () => {
14      const response = await axios.post(`${GATEWAY_URL}/auth/login`, {
15        username: 'admin',
16        password: 'password123'
17      });
18      expect(response.status).toBe(201);
19      expect(response.data.access_token).toBeDefined();
20      jwtToken = response.data.access_token; // on s'en souviendra pour les tests suivants
21    });
```

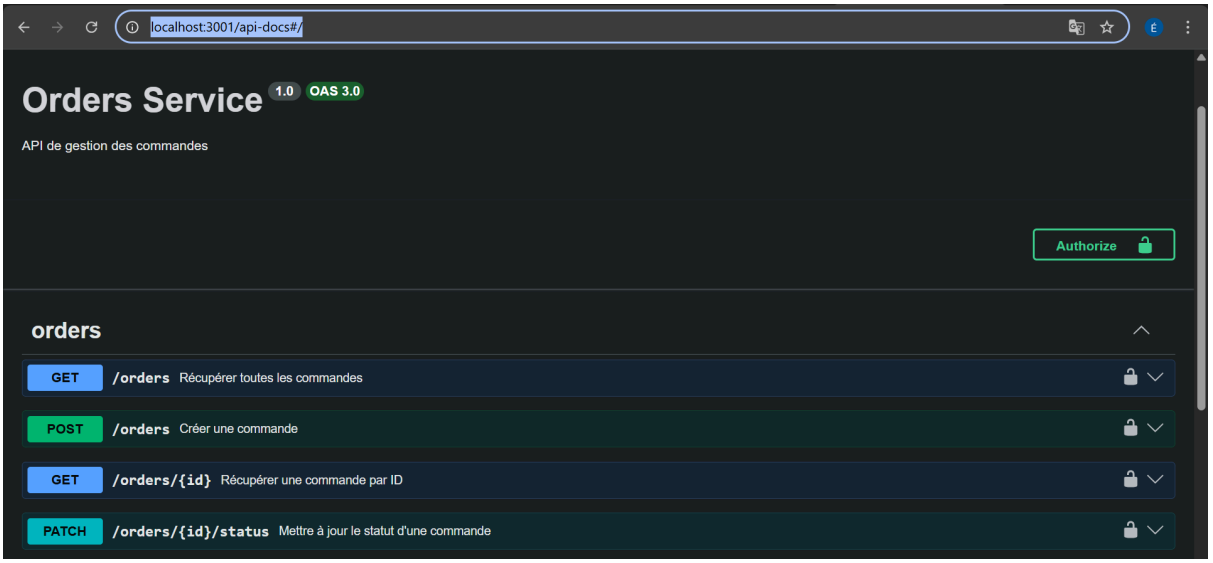
On lance nos services et exécute nos tests :

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS C:\Users\edoua\OneDrive - UPEC\Documents\BUT3\S6\R6.A.05 Développement Avancé\TP3\microservices-NestJS> cd .\api-gateway\
PS C:\Users\edoua\OneDrive - UPEC\Documents\BUT3\S6\R6.A.05 Développement Avancé\TP3\microservices-NestJS\api-gateway> npm run test

> api-gateway@0.0.1 test
> jest

 PASS  src/app.controller.spec.ts
 PASS  src/gateway.e2e.spec.ts

Test Suites: 2 passed, 2 total
Tests:       22 passed, 22 total
Snapshots:   0 total
Time:        1.26 s
Ran all test suites.
PS C:\Users\edoua\OneDrive - UPEC\Documents\BUT3\S6\R6.A.05 Développement Avancé\TP3\microservices-NestJS\api-gateway>

node orders...
node users-...
node api-ga...
powershell...
```

3 - Endpoint Swagger

On vérifie que tous nos endpoint apparaissent sur swagger :


localhost:3000/api-docs#/
users
GET /users Récupérer tous les utilisateurs
POST /users Créer un utilisateur
GET /users/{id} Récupérer un utilisateur par ID
PUT /users/{id} Modifier un utilisateur
DELETE /users/{id} Supprimer un utilisateur
auth
POST /auth/login Se connecter et récupérer un token JWT


localhost:3001/api-docs#/
Orders Service
1.0
OAS 3.0
API de gestion des commandes
Authorize
orders
GET /orders Récupérer toutes les commandes
POST /orders Créer une commande
GET /orders/{id} Récupérer une commande par ID
PATCH /orders/{id}/status Mettre à jour le statut d'une commande